

Ex-no: 10	FIRST FIT AND BEST FIT ALLOCATION
07/04/2026	

Aim

To implement and compare the **First Fit** and **Best Fit Memory Allocation Algorithms** for efficient allocation of memory blocks to processes.

Algorithm:

1. Start the program.
2. Input the number of memory blocks and their sizes.
3. Input the number of processes and their sizes.
4. Apply **First Fit Algorithm**:
 - For each process, scan memory blocks from the beginning.
 - Allocate the first block that is large enough.
 - Mark the block as allocated.
5. Apply **Best Fit Algorithm**:
 - For each process, search all memory blocks.
 - Allocate the smallest block that is sufficient for the process.
 - Mark the block as allocated.
6. If no suitable block is found, keep the process unallocated.
7. Display the allocation results for both algorithms.
8. Stop the program.

CODE:

FirstFit.c

```
// First Fit
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int m, n;
```

```
// =====
```

```
// INPUT NUMBER OF MEMORY BLOCKS
```

```
// =====
```

```
printf("Enter number of memory blocks:\n");
```

```
scanf("%d", &m);
```

```
/*
```

Example:

m = 5 blocks

Blocks:

B1, B2, B3, B4, B5

```
*/
```

```
// =====
```

```
// DECLARING BLOCK ARRAYS
```

```
// =====
```

```
int blockSize[m];    // stores current (remaining) size of each block
```

```
int originalBlockSize[m]; // stores original size for display purpose
```

```
// =====
```

```
// INPUT BLOCK SIZES
```

```
// =====
```

```
printf("Enter size of each block:\n");
```

```
/*
```

Example:

Block Sizes:

100 500 200 300 600

```
*/
```

```
for(int i = 0; i < m; i++) {
```

```
    scanf("%d", &blockSize[i]);
```

```
/*
```

Store original size separately

because blockSize[] will change

after allocation (splitting)

```
*/
```

```
    originalBlockSize[i] = blockSize[i];
```

```
}
```

```
// =====
```

```
// INPUT NUMBER OF PROCESSES
```

```
// =====
```

```
printf("Enter number of processes:\n");
```

```
scanf("%d", &n);
```

```
/*
```

Example:

n = 4 processes

Processes:

P1, P2, P3, P4

```
*/
```

```
// =====
```

```
// DECLARING PROCESS ARRAY
```

```
// =====
```

```
int processSize[n];
```

```
// =====
```

```
// INPUT PROCESS SIZES
```

```
// =====
```

```
printf("Enter size of each process:\n");
```

```
/*
```

Example:

Process Sizes:

212 417 112 426

```
*/
```

```
for(int i = 0; i < n; i++) {  
    scanf("%d", &processSize[i]);  
}
```

```
// =====
```

```
// FIRST FIT ALGORITHM STARTS
```

```
// =====
```

```
/*
```

First Fit Strategy:

- For each process, scan blocks from beginning
- Allocate the FIRST block that is large enough
- Reduce the block size (memory splitting)

```
*/
```

```

for(int i = 0; i < n; i++) {

    int allocated = 0;

    /*
    allocated = 1 → process allocated
    allocated = 0 → process not allocated
    */

    // =====
    // SEARCH BLOCKS SEQUENTIALLY
    // =====

    for(int j = 0; j < m; j++) {

        /*
        Condition to allocate:
        block must have enough space
        */

        if(blockSize[j] >= processSize[i]) {

            /*
            Internal Fragmentation:
            leftover space in that block

```

```
*/
```

```
int fragment = blockSize[j] - processSize[i];
```

```
// =====
```

```
// PRINT ALLOCATION DETAILS
```

```
// =====
```

```
printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
```

```
    i + 1, processSize[i], j + 1,
```

```
    originalBlockSize[j], fragment);
```

```
/*
```

```
Reduce block size after allocation
```

```
(remaining memory can be reused)
```

```
*/
```

```
blockSize[j] -= processSize[i];
```

```
allocated = 1;
```

```
/*
```

```
Break because First Fit uses
```

```
the FIRST suitable block only
```

```
*/
```

```

        break;
    }
}

// =====
// IF NO BLOCK FOUND
// =====

if(!allocated) {

    printf("Process %d of size %d is not allocated\n",
           i + 1, processSize[i]);

}

}

return 0;
}

```

BestFit.c:

```

// Best Fit

#include <stdio.h>

int main() {

```



```
int m, n;
```

```
// =====
```

```
// INPUT NUMBER OF MEMORY BLOCKS
```

```
// =====
```

```
printf("Enter number of memory blocks:\n");
```

```
scanf("%d", &m);
```

```
/*
```

Example:

m = 5 blocks

Blocks:

B1, B2, B3, B4, B5

```
*/
```

```
// =====
```

```
// DECLARING BLOCK ARRAYS
```

```
// =====
```

```
int blockSize[m]; // stores remaining size of each block
```

```
int original[m]; // stores original size for display
```

```
// =====
```

```
// INPUT BLOCK SIZES
```

```
// =====
```

```
printf("Enter size of each block:\n");
```

```
/*
```

Example:

Block Sizes:

100 500 200 300 600

```
*/
```

```
for(int i = 0; i < m; i++) {
```

```
    scanf("%d", &blockSize[i]);
```

```
/*
```

Save original block size

because blockSize[] will change

after allocation

```
*/
```

```
    original[i] = blockSize[i];
```

```
}
```

```
// =====
```

```
// INPUT NUMBER OF PROCESSES
```

```
// =====
```

```
printf("Enter number of processes:\n");
```

```
scanf("%d", &n);
```

```
/*
```

Example:

n = 4 processes

Processes:

P1, P2, P3, P4

```
*/
```

```
// =====
```

```
// DECLARING PROCESS ARRAY
```

```
// =====
```

```
int processSize[n];
```

```
// =====
```

```
// INPUT PROCESS SIZES
```

```
// =====
```

```
printf("Enter size of each process:\n");
```

```
/*
```

Example:

Process Sizes:

212 417 112 426

```
*/
```

```
for(int i = 0; i < n; i++) {  
    scanf("%d", &processSize[i]);  
}
```

```
// =====
```

```
// BEST FIT ALGORITHM STARTS
```

```
// =====
```

```
/*
```

Best Fit Strategy:

- For each process:

→ Check ALL memory blocks

→ Choose the block with

MINIMUM leftover space

- This reduces wastage

*/

```
for(int i = 0; i < n; i++) {
```

```
    int bestIndex = -1;
```

```
    /*
```

```
    bestIndex stores the index
```

```
    of the best suitable block
```

```
    -1 means no block found yet
```

```
    */
```

```
    // =====
```

```
    // SEARCH ALL BLOCKS
```

```
    // =====
```

```
    for(int j = 0; j < m; j++) {
```

```
        /*
```

```
        Check if block can fit process
```

```
        */
```

```

if(blockSize[j] >= processSize[i]) {

    /*
    Select block if:

    1. No block selected yet

    OR

    2. Current block is smaller
       than previously selected block
    */

    if(bestIndex == -1 || blockSize[j] < blockSize[bestIndex]) {
        bestIndex = j;
    }
}

// =====
// IF BEST BLOCK FOUND
// =====

if(bestIndex != -1) {

    /*
    Internal Fragmentation:
    leftover space after allocation

```

```
*/
```

```
int fragment = blockSize[bestIndex] - processSize[i];
```

```
// =====
```

```
// PRINT ALLOCATION DETAILS
```

```
// =====
```

```
printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
```

```
    i + 1, processSize[i], bestIndex + 1,
```

```
    original[bestIndex], fragment);
```

```
/*
```

```
Reduce block size after allocation
```

```
(remaining memory reused)
```

```
*/
```

```
blockSize[bestIndex] -= processSize[i];
```

```
}
```

```
// =====
```

```
// IF NO BLOCK FOUND
```

```
// =====
```

```
else {
```

```
        printf("Process %d of size %d is not allocated\n",  
              i + 1, processSize[i]);  
    }  
}  
  
return 0;  
}
```

OUTPUT:

Enter number of memory blocks: 5

Enter size of each block:

100 500 200 300 600

Enter number of processes: 4

Enter size of each process:

212 417 112 426

First Fit:

Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288

Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183

Process 3 of size 112 is allocated to Block 2 of size 212 with Fragment 100

Process 4 of size 426 is not allocated

Best Fit:

Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88

Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83

Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88

Process 4 of size 426 is allocated to Block 5 of size 600 with Fragment 174

Result:

Thus, the **First Fit** and **Best Fit Memory Allocation Algorithms** were implemented successfully, and memory blocks were allocated to processes based on different allocation strategies.